PURPLE TEAM ENGAGEMENT

Phase 1 Addendum

Detection Capability Re-Test

Operation Midnight Courier

APT29 / Cozy Bear (MITRE G0016)

Engagement	Operation Midnight Courier
Document Type	Phase 1 Addendum (Detection Re-Test)
Target Environment	corp.local (Dell T320 / Proxmox VE 9.1)
Threat Actor Emulated	APT29 / Cozy Bear (SVR, Russia)
Framework	MITRE ATT&CK G0016, CISA AA21-116A
Re-Test Date	26 April 2026
Original Engagement Date	19 April 2026
Document Version	1.0
Classification	Internal / Lab

Contents

1. Executive Summary	3
2. Background and Scope	4
3. Defensive Changes Deployed	5
4. Detection Tier Methodology	9
5. Re-Test Findings (Per Technique)	10
6. Detection Improvement Metrics	17
7. Identified Gaps and Phase 2 Backlog	18
8. Software Bill of Materials (SBOM)	20
9. Conclusion	22

1. Executive Summary

This addendum documents a focused detection capability re-test conducted seven days after the original Operation Midnight Courier engagement (Phase 1). The re-test isolates the impact of a single defensive change: the deployment of Sysmon endpoint logging via Group Policy and the integration of the resulting event channel into Wazuh SIEM.

Phase 1 found that 10 of 13 emulated APT29 techniques went entirely undetected by Wazuh, primarily due to the absence of endpoint process and registry telemetry. This addendum verifies whether the deployment of Sysmon closes that visibility gap, and where it does not, identifies the residual gaps for remediation in Phase 2.

Headline Results

Metric	Phase 1 (Original)	Phase 1 Addendum (Re-Test)
Techniques tested	13	7 (subset improvable by Sysmon)
Total disrupted (detected or blocked)	**3 of 13 (23%)**	**5 of 7 (71%)**
Tier 1 alerts (clean, MITRE-tagged)	0	2
Tier 2 (forensic only)	0	2
Tier 3 (missed)	10 of 13	2 of 7
Blocked at endpoint	0	**1 (Defender stopped LSASS dump)**

Key Findings

- **Sysmon deployment delivered a 3x improvement in attack chain disruption** (23% to 71%) from a single defensive change, validating the hypothesis from the Phase 1 report.
- **Two techniques produced clean Tier 1 alerts** with the MITRE technique ID auto-tagged at the rule level (T1059.001 Encoded PowerShell, T1547.001 Registry Run Keys).
- **LSASS credential dumping was blocked at the endpoint by Microsoft Defender**, however the corresponding Defender event log channel is not currently ingested by Wazuh, creating a visibility gap that will be remediated in Phase 2.
- **Two PowerShell cmdlet-based techniques remain undetected** (T1562.001 Defender disable, T1552.001 credential file read) because the activity occurs inside an existing PowerShell process and never spawns a new process. Remediation requires PowerShell Script Block Logging.
- **Wazuh rule severity tuning is required** for low-noise techniques such as T1018 Remote System Discovery, which fired at rule level 3 (informational) when targeting a Domain Controller warrants level 5 or higher.

2. Background and Scope

2.1 Phase 1 Recap

Operation Midnight Courier was a full APT29-emulated kill chain executed against the corp.local lab on 19 April 2026. The attack progressed from a GoPhish phishing lure delivered via hMailServer to oflynn@corp.local, through credential capture, RDP foothold on WIN11-01, SMB share enumeration discovering aturner Domain Admin credentials in a plaintext file, Kerberoasting of four service accounts, RDP pivot to DC01, Mimikatz LSASS dumping, DCSync extraction of the krbtgt NTLM hash, and finally Golden Ticket forgery valid for ten years.

The Phase 1 report concluded that Wazuh detected only 3 of 13 emulated techniques (23%), and that the dominant root cause was the absence of Sysmon on endpoints. Without Sysmon, the modular event types that drive most ATT&CK-aligned detections (process creation with full command line, registry value writes, file creates, image loads, and process access) were simply not being generated.

2.2 Scope of the Re-Test

This addendum is intentionally scoped narrowly. The objective is to isolate the effect of one defensive change (Sysmon deployment plus Wazuh ingestion of the Sysmon channel) and produce evidence-grade detection outcomes for inclusion in the engagement portfolio.

The following are out of scope for this re-test and are deferred to Phase 2:

- Kerberoasting (requires Windows Security audit policy tuning, not Sysmon)
- DCSync (requires audit policy on Domain Controller, not endpoint Sysmon)
- Network exfiltration detection (requires pfSense flow log integration)
- ADFS / AD CS based attacks (Phase 2 attack surface, not Phase 1)

2.3 Lab State at Re-Test

The lab was operated from the same misconfigured baseline used for Phase 1. The misconfigurations active during the re-test were:

- aturner promoted to Domain Admins
- svc_deploy promoted to Domain Admins
- WDigest UseLogonCredential set to 1 (cleartext credentials in LSASS)
- AD CS ESC6 enabled (EDITF_ATTRIBUTESUBJECTALTNAME2 CA-wide flag)
- AD CS ESC8 (HTTP web enrollment endpoint)
- IT_Resources SMB share with plaintext credential file IT_Creds.txt
- Four Kerberoastable service accounts (svc_sql, svc_backup, svc_web, svc_monitor)
- No Group Policy account lockout policy

3. Defensive Changes Deployed

This section documents every command executed during the deployment, the rationale behind it, and the expected outcome. The deployment was carried out from DC01 acting as the central distribution point. All Windows endpoints in scope (DC01, WIN11-01, WIN11-02) received the configuration via Group Policy.

3.1 Pre-Deployment Snapshot

Snapshots were taken on Proxmox before any changes to provide a rollback point. Although the existing clean-baseline-v2 snapshot from Phase 1 setup was still available, a fresh snapshot was created at the misconfigured-but-pre-Sysmon state to give us a faster recovery path if Sysmon deployment caused issues.

Command executed on Proxmox:

```
for i in 100 101 102; do qm snapshot $i pre-sysmon \
    --description "Before Sysmon rollout via GPO"; done
```

Why this matters: Sysmon deployment touches kernel-level driver loading and modifies system event channel subscriptions. While the operation is generally safe, having a snapshot before the change converts a potential rebuild scenario into a 30-second rollback. The cost is negligible (snapshots are copy-on-write and consume almost no disk until divergence).

3.2 Staging Sysmon Binary and Configuration in SYSVOL

SYSVOL is replicated automatically across all domain controllers and is readable by every domain-joined machine via the well-known UNC path. Placing the Sysmon binary and configuration here makes it accessible to the GPO startup script regardless of which workstation runs.

Commands executed on DC01 (PowerShell as Administrator):

```
# Create staging folder in SYSVOL
$sysmonPath = "\\corp.local\SYSVOL\corp.local\scripts\Sysmon"
New-Item -ItemType Directory -Path $sysmonPath -Force

# Download Sysmon from official Sysinternals source
Invoke-WebRequest `
    -Uri "https://download.sysinternals.com/files/Sysmon.zip" `
    -OutFile "C:\Temp\Sysmon.zip"

Expand-Archive -Path "C:\Temp\Sysmon.zip" `
    -DestinationPath "C:\Temp\Sysmon" -Force

Copy-Item "C:\Temp\Sysmon\Sysmon64.exe" `
    -Destination $sysmonPath -Force
```

Sysmon version deployed: v15.20 (Sysmon schema version 4.91), the current stable release at deployment time.

3.3 Pulling Olaf Hartong's Modular Sysmon Configuration

Two community-standard Sysmon configurations exist: the SwiftOnSecurity baseline (simpler, broad coverage) and Olaf Hartong's sysmon-modular (granular, MITRE-tagged at rule level). The modular config was selected for this lab because each rule embeds the corresponding MITRE technique ID directly into the RuleName field of the event, which means Wazuh sees pre-enriched events without needing custom rule mapping.

Command executed on DC01:

```
Invoke-WebRequest `
    -Uri "https://raw.githubusercontent.com/olafhartong/sysmon-
modular/master/sysmonconfig.xml" `
    -OutFile "$sysmonPath\sysmonconfig.xml"
```

Why this configuration: When a Sysmon event fires under this configuration, the RuleName field is populated with values such as `technique_id=T1547.001,technique_name=Registry Run Keys / Start Folder`. Wazuh ingests this verbatim, allowing analysts to filter, group, and report on events directly by ATT&CK technique without writing decoders.

3.4 Idempotent Install Script

The install script needs to handle two cases on each boot: a fresh machine where Sysmon is not yet installed, and a machine where Sysmon is already installed but the configuration may have been updated. The script below handles both, logs all actions to a local file for troubleshooting, and is safe to run repeatedly without side effects.

Script saved to \\corp.local\SYSVOL\corp.local\scripts\Sysmon\Install-Sysmon.ps1:

```
$sysmonShare = "\\corp.local\SYSVOL\corp.local\scripts\Sysmon"
$sysmonExe   = "$sysmonShare\Sysmon64.exe"
$sysmonConf  = "$sysmonShare\sysmonconfig.xml"
$localExe    = "C:\Windows\Sysmon64.exe"
$logFile     = "C:\Windows\Temp\sysmon-install.log"

function Write-Log($msg) {
    "$(Get-Date -Format 'yyyy-MM-dd HH:mm:ss') - $msg" |
        Out-File -FilePath $logFile -Append
}

Write-Log "=== Sysmon GPO deployment script started ==="
$svc = Get-Service -Name Sysmon64 -ErrorAction SilentlyContinue

if ($null -eq $svc) {
    Write-Log "Sysmon not installed. Installing..."
    Copy-Item $sysmonExe $localExe -Force
    & $localExe -accepteula -i $sysmonConf | Out-File -FilePath $logFile -Append
    Write-Log "Sysmon installed successfully."
} else {
    Write-Log "Sysmon already installed. Updating config..."
    & $localExe -c $sysmonConf | Out-File -FilePath $logFile -Append
    Write-Log "Sysmon config updated."
}

Write-Log "=== Sysmon GPO deployment script finished ==="
```

Behavior notes:

- On first boot the script copies Sysmon64.exe locally and runs the install with -accepteula -i pointing at the modular config.
- On subsequent boots the script detects the existing service and only refreshes the configuration via -c, which is a fast in-place update.
- The log file at C:\Windows\Temp\sysmon-install.log preserves a per-boot record for troubleshooting.

3.5 GPO Creation and Linking

A new Group Policy Object named Deploy-Sysmon was created and linked at the domain root so all current and future domain-joined Windows machines pick it up automatically.

Commands executed on DC01:

```
New-GPO -Name "Deploy-Sysmon" `
    -Comment "Deploys Sysmon via startup script from SYSVOL"

New-GPLink -Name "Deploy-Sysmon" `
    -Target "DC=corp,DC=local" -LinkEnabled Yes
```

The PowerShell startup script binding (Computer Configuration / Policies / Windows Settings / Scripts / Startup / PowerShell Scripts tab) was added via gpmc.msc, pointing at the SYSVOL location:

```
\\corp.local\SYSVOL\corp.local\scripts\Sysmon\Install-Sysmon.ps1
```

Important: GPO startup scripts run as the local SYSTEM account at boot time and are not subject to user-level PowerShell execution policy. This means the script runs without needing a code signature, even on workstations where execution policy is RemoteSigned.

3.6 Wazuh Channel Ingestion

Sysmon writes its events to a dedicated Windows event log channel called Microsoft-Windows-Sysmon/Operational. The Wazuh agent does not subscribe to this channel by default. Without explicit configuration, events generated by Sysmon never leave the endpoint.

The Wazuh approach used here is centralized via the shared agent.conf, which is pushed automatically to every Windows agent. This avoids per-host configuration drift.

Edits to /var/ossec/etc/shared/default/agent.conf on the Wazuh manager:

```
<agent_config os="Windows">
  <localfile>
    <location>Microsoft-Windows-Sysmon/Operational</location>
    <log_format>eventchannel</log_format>
  </localfile>
</agent_config>
```

Validation and apply commands:

```
sudo /var/ossec/bin/verify-agent-conf
sudo systemctl restart wazuh-manager
sudo /var/ossec/bin/agent_control -R -a
```

Note on the Defender channel: During the re-test, a separate event channel called Microsoft-Windows-Windows Defender/Operational was identified as a critical visibility source for blocked attacks. The Defender channel was deliberately not added to agent.conf in this addendum so that the gap could be measured cleanly. It will be added at the start of Phase 2.

3.7 Validation

Post-deployment validation confirmed:

- Sysmon64 service running on DC01, WIN11-01, and WIN11-02
- Microsoft-Windows-Sysmon/Operational event channel enabled on all three hosts
- Wazuh dashboard receiving events tagged with data.win.system.channel = "Microsoft-Windows-Sysmon/Operational"
- MITRE technique IDs visible in the data.win.eventdata.ruleName field of received events

4. Detection Tier Methodology

Detection results are not binary. An attack technique can produce a wide range of outcomes ranging from a clean alert that fires immediately to a forensic event that requires manual hunting. To capture this nuance, each technique in the re-test was classified into one of four tiers.

Tier 1	Real-time alert. Wazuh fires a rule with severity level 5 or higher. Event is MITRE-tagged at the rule name. SOC analyst sees this in their alert queue without effort.
Tier 2	Forensic only. Sysmon event is captured and reaches Wazuh, but no high-severity rule fires. Detection requires either manual hunting or rule tuning. Useful for incident reconstruction after the fact.
Tier 3	Missed entirely. No event captured at the relevant point in the kill chain. SIEM is blind. Remediation requires a different telemetry source or new logging policy.
Blocked	Attack stopped at the endpoint by Defender or another local control before completion. Whether the SIEM saw the block is tracked separately, since blocking and visibility are independent properties.

Two tiers can apply to the same technique when the kill chain has multiple phases. For example, T1552.001 in this re-test produced a Tier 2 detection on the recon phase (net view) and a Tier 3 miss on the credential read phase (Get-Content). Both are recorded.

5. Re-Test Findings

Each technique below documents the command executed, an explanation of what the command does, the expected detection signal, and the actual outcome. Commands were issued from WIN11-01 logged in as corp\Administrator over RDP, mirroring the Phase 1 attacker context.

5.1 T1059.001 Encoded PowerShell

Tactic	Execution
Phase 1 Result	**Tier 3 (missed entirely)**
Re-Test Result	**Tier 1 (clean alert with MITRE tag)**

Command executed:

```
$cmd = 'whoami /all; hostname; ipconfig /all'
$encoded = [Convert]::ToBase64String(
    [Text.Encoding]::Unicode.GetBytes($cmd))
powershell.exe -NoProfile -WindowStyle Hidden `
    -ExecutionPolicy Bypass -EncodedCommand $encoded
```

What this command does:

This is a textbook APT29 evasion pattern. The attacker assembles a string containing the actual commands they want to run, encodes it as Base64 (UTF-16LE, which is what -EncodedCommand expects), and passes the Base64 blob to a new powershell.exe instance with three suspicious flags layered together: -NoProfile (skip user profile to avoid logging hooks), -WindowStyle Hidden (no visible window), and -ExecutionPolicy Bypass (override script restrictions). Each flag individually is sometimes legitimate; the combination of all three plus -EncodedCommand is overwhelmingly used by malware and red teams.

Detection outcome:

Sysmon Event ID 1 (Process Creation) fired with the full deobfuscated command line visible. Wazuh ingested the event tagged with technique_id=T1059.001. The full process tree was captured (parent powershell.exe spawning child powershell.exe with the encoded payload), along with file hashes (SHA1, MD5, SHA256, IMPHASH) for the powershell.exe binary itself. This level of detail is investigation-grade and would be sufficient for a SOC analyst to confirm the technique without leaving the dashboard.

5.2 T1053.005 Scheduled Task Persistence

Tactic	Persistence
Phase 1 Result	**Tier 3 (missed entirely)**
Re-Test Result	**Tier 2 (forensic only, no Wazuh rule fired)**

Command executed:

```
schtasks /create `
    /tn "\Microsoft\Windows\WindowsUpdate\AUCheck" `
    /tr "powershell.exe -WindowStyle Hidden -Command Write-Host 'persistence test'" `
    /sc DAILY /st 08:00 /ru SYSTEM /f
```

What this command does:

Creates a scheduled task that runs daily at 08:00 under the SYSTEM account. The task name and folder path are deliberately chosen to mimic a legitimate Windows Update task. The /ru SYSTEM flag tells the scheduler to run the task as NT AUTHORITY\SYSTEM, which gives the persistence mechanism full local privileges. APT29 favors this technique because masquerading inside the Windows Update task tree makes the persistence invisible to casual review of the Task Scheduler GUI.

Detection outcome:

Sysmon captured Event ID 7 (Image Loaded) for taskschd.dll being loaded by taskhostw.exe and schtasks.exe, but did not fire a high-severity Wazuh rule tagged with T1053.005. The forensic trail exists (an analyst hunting for scheduled task creation could find evidence) but no real-time alert was generated. Closing this gap requires either a Wazuh custom rule matching schtasks.exe command lines or, preferably, enabling Windows Security audit policy for Other Object Access Events to receive Event ID 4698 (Scheduled task created) with full task XML.

5.3 T1547.001 Registry Run Key Persistence

Tactic	Persistence, Privilege Escalation
Phase 1 Result	**Tier 3 (missed entirely)**
Re-Test Result	**Tier 1 (clean alert with MITRE tag)**

Command executed:

```
reg add "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" `
    /v "WindowsSecurityUpdate" /t REG_SZ `
    /d "powershell.exe -WindowStyle Hidden -Command Start-Sleep 10" /f
```

What this command does:

Writes a value to the HKCU Run key, which causes Windows to launch the specified command every time the current user logs on. The value name WindowsSecurityUpdate is chosen to look legitimate. The payload is a hidden PowerShell process; in a real engagement this would be a download cradle pulling a second-stage payload from a C2 server. The /f flag forces overwrite without prompting, useful in scripted attacks.

Detection outcome:

This was the strongest detection of the re-test. Sysmon Event ID 13 (RegistryValueSet) fired with the complete payload visible in data.win.eventdata.details. The rule name field carried technique_id=T1547.001,technique_name=Registry Run Keys / Start Folder, the user CORP\Administrator was identified, the parent process chain (powershell.exe to reg.exe) was captured, and a corresponding Wazuh rule fired. A SOC analyst would have everything needed to investigate within a single event view.

5.4 T1562.001 Disable Windows Defender

Tactic	Defense Evasion
Phase 1 Result	**Tier 3 (missed entirely)**
Re-Test Result	**Tier 3 (still missed, root cause identified)**

Commands executed:

```
Set-MpPreference -DisableRealtimeMonitoring $true
Add-MpPreference -ExclusionPath "C:\Windows\Temp"
```

What these commands do:

The first cmdlet disables real-time scanning, which is the engine that inspects file writes, process loads, and script execution as they happen. The second adds a path exclusion for C:\Windows\Temp, telling Defender to ignore anything under that folder. Together these two changes carve out a safe zone where the attacker can stage tooling without scrutiny. APT29 uses this pattern routinely before deploying any binary that might trip a signature.

Detection outcome:

Both commands succeeded on the host (verified via Get-MpPreference) but produced no events in Wazuh. The root cause is that Set-MpPreference and Add-MpPreference are PowerShell cmdlets which execute inside the existing powershell.exe process. No new process is created, so Sysmon Event ID 1 does not fire. Olaf Hartong's modular configuration also does not currently include a registry-based watch for the HKLM\Software\Microsoft\Windows Defender* hives.

Remediation: Enable PowerShell Script Block Logging (GPO setting Computer Configuration / Administrative Templates / Windows Components / Windows PowerShell / Turn on PowerShell Script Block Logging). This produces Event ID 4104 in the Microsoft-Windows-PowerShell/Operational channel for every script block executed, including cmdlets running inside an existing session. Sysmon-only telemetry will never catch this class of attack.

5.5 T1018 + T1552.001 Recon and Credential File Access

Tactic	Discovery, Credential Access
Phase 1 Result	**Tier 3 (entire chain missed)**
Re-Test Result (recon)	**Tier 2 (rule level 3, low severity)**
Re-Test Result (file read)	**Tier 3 (no event captured)**

Commands executed:

```
net view \\dc01.corp.local
dir \\dc01.corp.local\IT_Resources\
Get-Content \\dc01.corp.local\IT_Resources\IT_Creds.txt
Copy-Item \\dc01.corp.local\IT_Resources\IT_Creds.txt `
    -Destination C:\Windows\Temp\creds_loot.txt
```

What these commands do:

This four-step sequence emulates the post-foothold credential discovery workflow used in Phase 1. First, net view enumerates SMB shares exposed by the Domain Controller to identify accessible network locations. Second, dir lists the contents of the IT_Resources share. Third, Get-Content reads the plaintext credential file (which contains aturner's Domain Admin password from the Phase 1 misconfig). Fourth, Copy-Item stages the file locally under C:\Windows\Temp for later exfiltration. The attacker now possesses a Domain Admin credential and can pivot to DC01.

Detection outcome:

The recon phase (net view) fired Sysmon Event ID 1 for net.exe with the parent process chain visible and the modular config tagged the event as technique_id=T1018,technique_name=Remote System Discovery. Wazuh fired rule 92033 (Discovery activity spawned via powershell execution) but only at level 3, which is informational severity. In a busy SOC, this would sit in the noise floor.

The credential file read and local copy phases (Get-Content and Copy-Item) generated no events. Both are PowerShell cmdlets running inside the existing PowerShell session, hitting the same root cause as T1562.001. The Sysmon Event ID 11 (FileCreate) that should have fired for the local copy to C:\Windows\Temp also did not appear, which suggests the modular config's FileCreate watch list does not include plain .txt extensions in the Temp folder.

Remediation: Two changes are required. First, raise the Wazuh rule severity for T1018 events when the target is a Domain Controller (custom rule). Second, enable PowerShell Script Block Logging to catch the cmdlet activity, and consider extending Sysmon's FileCreate scope to cover sensitive local paths.

5.6 T1003.001 LSASS Memory Dump via comsvcs.dll

Tactic	Credential Access
Phase 1 Result	**Tier 3 (missed; Defender was disabled in Phase 1)**
Re-Test Result (endpoint)	**Blocked by Defender**
Re-Test Result (SIEM)	**Tier 3 (Defender event channel not ingested)**

Command executed:

```
$lsassPid = (Get-Process lsass).Id
rundll32.exe C:\Windows\System32\comsvcs.dll, MiniDump `
    $lsassPid C:\Windows\Temp\lsass.dmp full
```

What this command does:

This is APT29's signature living-off-the-land credential dumping technique. comsvcs.dll is a legitimate Microsoft DLL shipped with every Windows installation. It exports a MiniDump function that creates a memory dump of any process the caller has access to. By invoking it through rundll32 (also a legitimate Windows binary) with the LSASS process ID, the attacker uses two trusted Microsoft-signed tools to produce a full memory image of LSASS, which contains NTLM hashes, Kerberos tickets, and (with WDigest enabled) cleartext passwords. No third-party tooling is dropped, no unsigned binary executes, and the technique often slips past signature-based defenses.

Detection outcome:

Microsoft Defender detected the command line at process creation and blocked rundll32 from executing. The lsass.dmp file was never written. Defender's local event log (Microsoft-Windows-Windows Defender/Operational) recorded Event ID 1116 (Warning, malware detected) and Event ID 1117 (action taken) with the threat name HackTool:Win32/DumpLsass.H and the full attack command line preserved.

Critically, none of these Defender events reached Wazuh. The Defender Operational channel is not currently included in the agent.conf localfile subscriptions. The endpoint successfully defended itself, but the SIEM has no record that an attack was attempted or blocked. From a SOC perspective, this is invisible.

Remediation: Add the Microsoft-Windows-Windows Defender/Operational channel to the Wazuh shared agent.conf. This is a five-minute change that will unlock visibility into every Defender block, quarantine, and configuration change across the estate. Scheduled for the start of Phase 2.

6. Detection Improvement Metrics

6.1 Per-Technique Comparison

#	MITRE	Technique	Phase 1	Re-Test	Tier
1	T1059.001	Encoded PowerShell	Missed	**Detected**	**Tier 1**
2	T1053.005	Scheduled Task	Missed	Forensic only	**Tier 2**
3	T1547.001	Registry Run Key	Missed	**Detected**	**Tier 1**
4	T1562.001	Defender Disable	Missed	Missed	**Tier 3**
5a	T1018	Net View Recon	Missed	Forensic only	**Tier 2**
5b	T1552.001	Credential File Read	Missed	Missed	**Tier 3**
6	T1003.001	LSASS Memory Dump	Missed	**Blocked at endpoint**	**Blocked**

6.2 Aggregate Outcomes

- **Total disrupted (detected at any tier or blocked):** 5 of 7 techniques (71%), up from 0 of these same 7 in Phase 1.
- **Tier 1 clean alerts:** 2 of 7 (29%), up from 0 in Phase 1.
- **Tier 2 forensic-only:** 2 of 7 (29%), all attributable to either Wazuh rule severity tuning or Sysmon FileCreate scope.
- **Tier 3 missed:** 2 of 7 (29%), both rooted in the same cause: PowerShell cmdlets executing inside an existing process. Single remediation (Script Block Logging) closes both.
- **Blocked at endpoint:** 1 of 7. Defender prevented LSASS dumping. SIEM lacks visibility into the block, scheduled for Phase 2 fix.

7. Identified Gaps and Phase 2 Backlog

The re-test surfaced four concrete gaps. Each is presented with the impact, the remediation, and the priority for Phase 2.

7.1 Gap: Defender Operational Channel Not Ingested

Impact: Endpoint blocks of malicious activity are invisible to the SIEM. The LSASS dump prevention in this re-test produced rich Defender events on the host that Wazuh never saw. In a real engagement, blocked attacks would not appear in the alert queue, removing the SOC's ability to triage active threats or build threat hunts from observed adversary behavior.

Remediation: Add Microsoft-Windows-Windows Defender/Operational to the Wazuh shared agent.conf as an eventchannel localfile. Push to all Windows agents.

Priority: High. Effort: 5 minutes. Action: scheduled as the first task of Phase 2.

7.2 Gap: PowerShell Script Block Logging Not Enabled

Impact: Two of the seven re-test techniques (T1562.001 Defender disable and T1552.001 credential file read) executed entirely as PowerShell cmdlets inside an existing PowerShell process. No new process spawned, so Sysmon Event ID 1 cannot detect them. This is a structural blind spot for Sysmon-only telemetry.

Remediation: Deploy a Group Policy enabling PowerShell Script Block Logging (Computer Configuration / Administrative Templates / Windows Components / Windows PowerShell / Turn on PowerShell Script Block Logging). Subscribe Wazuh agents to the Microsoft-Windows-PowerShell/Operational channel. Tune Wazuh rules to alert on script block content matching known suspicious patterns (Set-MpPreference -DisableRealtimeMonitoring, Get-MpPreference reads, Add-MpPreference exclusions, Get-Content of files in admin shares).

Priority: High. Effort: 30 minutes (GPO creation, Wazuh channel subscription, basic rule tuning). Action: scheduled for early Phase 2.

7.3 Gap: Wazuh Rule Severity Tuning

Impact: T1018 Remote System Discovery from a domain workstation against a Domain Controller fired at level 3 (informational). In a busy SOC this would be filtered out as noise, despite being a clear indicator of post-compromise reconnaissance.

Remediation: Author a custom Wazuh rule that elevates the severity of net.exe enumeration commands when the target is a Domain Controller hostname. Apply similar treatment to other Discovery techniques (T1087 Account Discovery, T1069 Permission Groups Discovery, T1482 Domain Trust Discovery) and chain them so a sequence of Discovery events from a single host within a short window fires a higher-severity correlation rule.

Priority: Medium. Effort: 1 to 2 hours of rule authoring and testing. Action: scheduled mid-Phase 2.

7.4 Gap: Domain Controller Audit Policy

Impact: Two of the most consequential APT29 techniques in the original kill chain (Kerberoasting, DCSync) cannot be detected by endpoint Sysmon because they manifest as Kerberos and directory-replication operations on the DC, not as endpoint process activity. Phase 1 missed both entirely.

Remediation: Configure DC01 audit policy to enable: Audit Kerberos Service Ticket Operations (catches 4769 events for Kerberoasting), Audit Directory Service Access (catches 4662 for DCSync replication), and Audit Other Object Access Events (catches 4698 for scheduled task creation). Verify Wazuh is subscribed to the Security channel and that rules exist or are added for these event IDs.

Priority: Medium. Effort: 1 hour. Action: prerequisite for Phase 2 emulation, must be in place before re-running the credential access portion of the kill chain.

8. Software Bill of Materials (SBOM)

This section enumerates every component, tool, and external dependency used to produce the defensive changes in this addendum. The intent is reproducibility and supply-chain transparency for the lab environment.

8.1 Endpoint Detection Components

Component	Version	Vendor	Purpose	Source
Sysmon	v15.20	Microsoft Sysinternals	Endpoint event logging (process, registry, file, image, network)	download.sysinternals.com
Sysmon driver	Schema 4.91	Microsoft Sysinternals	Kernel-level event capture	Bundled with Sysmon
sysmon-modular config	master branch (April 2026)	Olaf Hartong (community)	MITRE-tagged Sysmon ruleset	github.com/olafhartong/sysmon-modular
Microsoft Defender Antivirus	Engine 1.1.26030.3008	Microsoft	Endpoint protection, signatures, AMSI	Built-in Windows 11
Defender Security Intelligence	AV/AS 1.449.311.0	Microsoft	Threat signatures (HackTool:Win32/DumpLsass.H)	Microsoft Update

8.2 SIEM and Log Management

Component	Version	Vendor	Purpose	Source
Wazuh Manager	4.14.4	Wazuh Inc.	Central event ingestion, rule engine, agent control	packages.wazuh.com
Wazuh Indexer	4.14.4 (OpenSearch)	Wazuh Inc.	Event storage and search	Bundled with Wazuh AIO
Wazuh Dashboard	4.14.4 (OpenSearch Dashboards)	Wazuh Inc.	Visualization and analyst UI	Bundled with Wazuh AIO
Wazuh Agent (Windows)	4.14.4	Wazuh Inc.	Endpoint log forwarder	packages.wazuh.com

8.3 Operating Systems

Host	OS	Build	Role
DC01	Windows Server 2022	Standard, Desktop Experience	Domain Controller, AD CS, GPO source
WIN11-01	Windows 11 Pro	10.0.26100.7019 (25H2)	Endpoint, primary attack target
WIN11-02	Windows 11 Pro	10.0.26100.7019 (25H2)	Endpoint, lateral movement target
WAZUH	Ubuntu Server	22.04.5 LTS	SIEM host
Hypervisor	Proxmox VE	9.1 (kernel 6.17.2-1-pve)	Virtualization platform

8.4 External Sources Touched During Deployment

URL	Purpose	Trust Anchor
https://download.sysinternals.com/files/Sysmon.zip	Sysmon binary	Microsoft (Sysinternals)
https://raw.githubusercontent.com/olafhartong/sysmon-modular/master/sysmonconfig.xml	Sysmon ruleset	Olaf Hartong (community)
https://packages.wazuh.com/	Wazuh agent and manager packages	Wazuh Inc.

8.5 Threat Intelligence Sources Referenced

- MITRE ATT&CK Framework, group profile G0016 (APT29 / Cozy Bear)
- CISA Cybersecurity Advisory AA21-116A (APT29 SVR exploitation activity)
- CISA Cybersecurity Advisory AA22-011A (APT29 multi-vector campaign)
- CISA Cybersecurity Advisory AA23-347A (Midnight Blizzard ongoing operations)
- MITRE ATT&CK Adversary Emulation Plan: APT29

9. Conclusion

The defensive change documented in this addendum (Sysmon GPO deployment with Wazuh ingestion of the Sysmon channel) delivered a measurable and significant improvement to the lab's detection posture. Of the seven techniques re-tested, five were either detected or blocked, compared to zero in the original Phase 1 engagement covering the same techniques.

The improvement was not uniform. Two techniques produced clean Tier 1 alerts that any SOC analyst would surface immediately. Two more produced forensic-grade evidence that requires either rule tuning or analyst hunting to operationalize. Two remained completely invisible, both because of the same root cause (PowerShell cmdlets running inside an existing process), and one was blocked at the endpoint by Defender but is invisible to the SIEM because the relevant event channel is not yet ingested.

This nuance is the core value of purple team exercises. A paper-only review would have logged Sysmon as a successful improvement and moved on. The actual emulation surfaced the structural limits of process-creation telemetry, the importance of script block logging as a complement, the visibility gap created by missing event channel subscriptions, and the need for severity tuning on Discovery-class events. None of these would have been visible without running the actual attack.

Phase 2 will close the four identified gaps before extending the scope to AD CS certificate-based privilege escalation (ESC1, ESC4, ESC8) and ADFS Golden SAML attacks. The Phase 2 starting baseline will include all four remediations from this addendum.

End of Phase 1 Addendum.

Next: Phase 2 (AD CS ESC + ADFS Golden SAML) to follow.